



Evaluacion 2021s1 - Parcial

Estructura de datos (Universidad Nacional de Quilmes)



Scan to open on Studocu

Evaluación integradora – 2021 - Estructuras de Datos – UNQ

Aclaraciones:

- Esta evaluación es a libro abierto. Se pueden usar todas las funciones y tipos de datos vistos en la práctica y en la teórica, salvo que el enunciado indique lo contrario.
- Recuerde respetar el formato del mail con el que se entrega la solución al examen.
- Respire hondo, tome pausas cuando corresponda, y recuerde que de cometer errores se aprende.

Ejercicio 1 Se cuenta con el siguiente tipo algebraico:

```
data Tree a = NodeT a (Tree a) (Tree a)
             | EmptyT
```

Definir las siguientes funciones **utilizando recursión estructural explícita** e **indicar el costo** obtenido para cada solución dada:

a) `conLaSumaCalculada :: Tree Int -> Tree Int`

Propósito: dado un árbol binario computa un árbol con misma estructura, pero en el que cada nodo posee la suma de los elementos de todo su subárbol (incluyendo al elemento en su raíz).

Precondición: no tiene.

Ejemplo:

```
sumaCalculada (NodeT 1 (NodeT 2 EmptyT EmptyT)
                      (NodeT 3
                        (NodeT 4 EmptyT EmptyT)
                        (NodeT 5 EmptyT EmptyT)))
->
NodeT 14 (NodeT 2 EmptyT EmptyT)
      (NodeT 12
        (NodeT 4 EmptyT EmptyT)
        (NodeT 5 EmptyT EmptyT))
```

b) `caminoQueSumaMas :: Tree Int -> [Int]`

Propósito: dado un árbol devuelve los elementos del camino cuya suma es mayor al resto de los caminos del árbol. Los caminos van desde la raíz hacia las hojas.

Precondición: no tiene.

Ejemplo:

```
caminoQueSumaMas (NodeT 1 (NodeT 2 EmptyT EmptyT)
                       (NodeT 3
                         (NodeT 4 EmptyT EmptyT)
                         (NodeT 5 EmptyT EmptyT)))
->
[1, 3, 5]
```

Ejercicio 2 Tenemos la siguiente interfaz del TAD `Alien`, donde el costo de todas las operaciones es $O(1)$:

■ `nuevoAlien :: String -> Alien`

Propósito: devuelve un alien que posee la raza dada y que tiene cero habilidades.

■ `habilidades :: Alien -> [String]`

Propósito: devuelve la lista de habilidades que ese alien posee.

■ `cantHabilidades :: Alien -> Int`

Propósito: devuelve la cantidad de habilidades que ese alien posee.

■ `raza :: String -> String`
Propósito: devuelve la raza del alien.

■ `aprender :: String -> Alien -> Alien`
Propósito: agrega una habilidad al alien.

Y contamos con al siguiente representación para un TAD *Galaxia*:

```
data Galaxia = ConsG (Map String (PriorityQueue Alien)) (Map String [String])
```

De esta representación sabemos que

- el primer `Map` relaciona razas de aliens con aliens ordenados en base a la cantidad de habilidades que poseen;
- el segundo `Map` relaciona habilidades con las razas que poseen dicha habilidad.

Definir las siguientes funciones como implementador del TAD *Galaxia*, **indicando el costo obtenido**:

a) `elMasHabilidosoEntre :: [String] -> Galaxia -> [Alien]`

Propósito: dada una lista de razas devuelve al alien más habilidoso entre dichas razas.

Precondición: existe al menos un alien para cada raza dada.

b) `enseñarARaza :: String -> String -> Galaxia -> Galaxia`

Propósito: dada una habilidad y una raza enseña dicha habilidad a todos los alien de dicha raza.

Precondición: la raza existe, pero los alien de esa raza aún no tienen dicha habilidad.

Nota: la habilidad puede o no ya existir.

Anexo de interfaces

`PriorityQueue`, siendo P la cantidad de elementos en la estructura:

<code>emptyPQ :: PriorityQueue a</code>	$O(1)$
<code>isEmptyPQ :: PriorityQueue a -> Bool</code>	$O(1)$
<code>insertPQ :: Ord a => a -> PriorityQueue a -> PriorityQueue a</code>	$O(\log P)$
<code>maxPQ :: PriorityQueue a -> a</code>	$O(1)$
<code>deleteMaxPQ :: Ord a => PriorityQueue a -> PriorityQueue a</code>	$O(\log P)$

`Map`, siendo K la cantidad de claves distintas en el map:

<code>emptyM :: Map k v</code>	$O(1)$
<code>assocM :: Ord k => k -> v -> Map k v -> Map k v</code>	$O(\log K)$
<code>lookupM :: Ord k => k -> Map k v -> Maybe v</code>	$O(\log K)$
<code>deleteM :: Ord k => k -> Map k v -> Map k v</code>	$O(\log K)$
<code>domM :: Map k v -> [k]</code>	$O(K)$